

# PRODUCT REVIEW AND RATING MANAGEMENT SYSTEM

## 1. Database Design

### 1.1 Review Table

| Attribute   | Data Type     | Constraint  |
|-------------|---------------|-------------|
| Review_ID   | NUMBER        | Primary Key |
| Customer_ID | NUMBER        | Foreign Key |
| Product_ID  | NUMBER        | Foreign Key |
| Review_Text | VARCHAR2(500) | NOT NULL    |
| Review_Date | DATE          | NOT NULL    |

### 1.2 Rating Table

| Attribute   | Data Type | Constraint  |
|-------------|-----------|-------------|
| Rating_ID   | NUMBER    | Primary Key |
| Customer_ID | NUMBER    | Foreign Key |
| Product_ID  | NUMBER    | Foreign Key |
| Rating      | NUMBER    | NOT NULL    |
| Rating_Date | DATE      | NOT NULL    |

## 2. Table Relationships

### Customer → Review

A customer can write **many reviews**.

### Product → Review

A product can receive **many reviews**.

### Customer → Rating

A customer can give **many ratings**.

### Product → Rating

A product can receive **many ratings**.

## 3. Table Creation

### 3.1 Create Review Table

```
SQL> CREATE TABLE Review (  
  2     Review_ID NUMBER PRIMARY KEY,  
  3     Customer_ID NUMBER,  
  4     Product_ID NUMBER,  
  5     Review_Text VARCHAR2(500) NOT NULL,  
  6     Review_Date DATE NOT NULL,  
  7     FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),  
  8     FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID)  
  9 );  
  
Table created.
```

### 3.2 Create Rating Table

```
SQL> CREATE TABLE Rating (  
  2     Rating_ID NUMBER PRIMARY KEY,  
  3     Customer_ID NUMBER REFERENCES Customer(Customer_ID),  
  4     Product_ID NUMBER REFERENCES Product(Product_ID),  
  5     Rating NUMBER NOT NULL,  
  6     Rating_Date DATE NOT NULL  
  7 );  
  
Table created.
```

## 4. Review Data Insertion

```
SQL> INSERT INTO Review VALUES
  2 (1, 1, 101, 'Good quality and comfortable kurti', DATE '2026-10-01');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (2, 2, 102, 'Nice shirt and good fitting', DATE '2026-10-02');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (3, 3, 103, 'Good material for kids', DATE '2026-10-03');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (4, 4, 104, 'Very comfortable jeans', DATE '2026-10-04');

1 row created.

SQL>
SQL> INSERT INTO Review VALUES
  2 (5, 5, 105, 'Good shoes for daily use', DATE '2026-10-05');

1 row created.
```

## 5. Rating Data Insertion

```
SQL> INSERT INTO Rating
  2 VALUES (1, 1, 101, 5, DATE '2026-10-01');

1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (2, 2, 102, 4, DATE '2026-10-02');

1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (3, 3, 103, 4, DATE '2026-10-03');

1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (4, 4, 104, 5, DATE '2026-10-04');

1 row created.

SQL>
SQL> INSERT INTO Rating
  2 VALUES (5, 5, 105, 5, DATE '2026-10-05');

1 row created.
```

## 6. Retrieve Product Review Details

```
SQL> SELECT
  2     p.Product_Name,
  3     c.Customer_Name,
  4     r.Review_Text,
  5     r.Review_Date
  6 FROM Review r
  7 JOIN Customer c
  8 ON r.Customer_ID = c.Customer_ID
  9 JOIN Product p
 10 ON r.Product_ID = p.Product_ID;
```

PRODUCT\_NAME

CUSTOMER\_NAME

REVIEW\_TEXT

REVIEW\_DA

Women Kurti  
Keerthiga  
Good quality and comfortable kurti  
01-OCT-26

PRODUCT\_NAME

CUSTOMER\_NAME

REVIEW\_TEXT

REVIEW\_DA

REVIEW\_DA

-----

Men Shirt

Keerthana

Nice shirt and good fitting

02-OCT-26

PRODUCT\_NAME

-----

CUSTOMER\_NAME

-----

REVIEW\_TEXT

-----

REVIEW\_DA

-----

Kids T-Shirt

Mythili

Good material for kids

03-OCT-26

PRODUCT\_NAME

-----

CUSTOMER\_NAME

-----

REVIEW\_TEXT

-----

REVIEW\_DA

-----

Women Jeans

Harini

Very comfortable jeans

04-OCT-26

## 7. Calculate Average Product Ratings

```
SQL> SELECT
  2     p.Product_Name,
  3     AVG(r.Rating) AS Average_Rating
  4 FROM Rating r
  5 JOIN Product p
  6 ON r.Product_ID = p.Product_ID
  7 GROUP BY p.Product_Name;
```

```
PRODUCT_NAME
```

```
-----
```

```
AVERAGE_RATING
```

```
-----
```

```
Women Kurti
              5
```

```
Men Shirt
           4
```

```
Kids T-Shirt
           4
```

```
PRODUCT_NAME
```

```
-----
```

```
AVERAGE_RATING
```

```
-----
```

```
Women Jeans
              5
```

```
Sports Shoes
           5
```

## 8. Identify Highly Rated Products

```
SQL> SELECT
  2     p.Product_Name,
  3     AVG(r.Rating) AS Average_Rating
  4 FROM Rating r
  5 JOIN Product p
  6 ON r.Product_ID = p.Product_ID
  7 GROUP BY p.Product_Name
  8 HAVING AVG(r.Rating) >= 4;
```

PRODUCT\_NAME

AVERAGE\_RATING

Women Kurti  
5

Men Shirt  
4

Kids T-Shirt  
4

PRODUCT\_NAME

AVERAGE\_RATING

Women Jeans  
5

Sports Shoes

## 9. Product Rating Report

```

SQL> SELECT
  2     p.Product_ID,
  3     p.Product_Name,
  4     COUNT(r.Rating_ID) AS Total_Ratings,
  5     AVG(r.Rating) AS Average_Rating
  6 FROM Product p
  7 JOIN Rating r
  8 ON p.Product_ID = r.Product_ID
  9 GROUP BY
 10     p.Product_ID,
 11     p.Product_Name
 12 ORDER BY
 13     p.Product_ID;

```

PRODUCT\_ID

-----

PRODUCT\_NAME

-----

TOTAL\_RATINGS AVERAGE\_RATING

-----

|             |   |   |
|-------------|---|---|
| 101         |   |   |
| Women Kurti | 1 | 5 |

|           |   |   |
|-----------|---|---|
| 102       |   |   |
| Men Shirt | 1 | 4 |

PRODUCT\_ID

-----

PRODUCT\_NAME

-----

TOTAL\_RATINGS AVERAGE\_RATING

-----

```
      102
Men Shirt
      1          4
```

```
PRODUCT_ID
-----
```

```
PRODUCT_NAME
-----
```

```
TOTAL_RATINGS  AVERAGE_RATING
-----
```

```
      103
Kids T-Shirt
      1          4
```

```
      104
Women Jeans
```

```
PRODUCT_ID
-----
```

```
PRODUCT_NAME
-----
```

```
TOTAL_RATINGS  AVERAGE_RATING
-----
```

```
      1          5
```

```
      105
Sports Shoes
      1          5
```